

[Início](#) > [Tutoriais](#) > [Python](#)

Classificação de floresta aleatória com o Scikit-Learn

Este artigo aborda como e quando você deve usar a classificação Random Forest com o scikit-learn. Com foco em conceitos, fluxo de trabalho e exemplos. Também abordamos como usar a matriz de confusão e as importâncias dos recursos.

Atualizado 11 de set. de 2024 · 14 min lido

Este tutorial explica como usar florestas aleatórias para classificação em Python. Abordaremos o assunto:

- Como funcionam as florestas aleatórias
- Como usá-los para classificação
- Como avaliar seu desempenho

Para aproveitar ao máximo este artigo, você deve ter um conhecimento básico de Python, pandas e scikit-learn. É útil entender como as árvores de decisão são usadas para classificação, portanto, considere ler primeiro o [Tutorial de Classificação de Árvores de Decisão em Python](#). Se você está apenas começando a usar o scikit-learn, consulte [Kaggle Tutorial: Seu primeiro modelo de aprendizado de máquina](#).

Embora as florestas aleatórias possam ser usadas tanto para classificação quanto para regressão, este artigo se concentrará na criação de um modelo de classificação. Para experimentar facilmente o código deste tutorial, [visite a pasta de trabalho do DataLab que o acompanha](#).

Pratique a classificação Random Forest com o Scikit-Learn neste exercício prático.

Uma visão geral das florestas aleatórias

As florestas aleatórias são um algoritmo popular de aprendizado de máquina supervisionado.

- As florestas aleatórias são para aprendizado de máquina supervisionado, em que há uma variável-alvo rotulada.
- As florestas aleatórias podem ser usadas para resolver problemas de regressão (variável-alvo numérica) e classificação (variável-alvo categórica).
- As florestas aleatórias são um método de conjunto, o que significa que elas combinam previsões de outros modelos.
- Cada um dos modelos menores no conjunto de florestas aleatórias é uma árvore de decisão.

Aprenda R para aprendizado de máquina

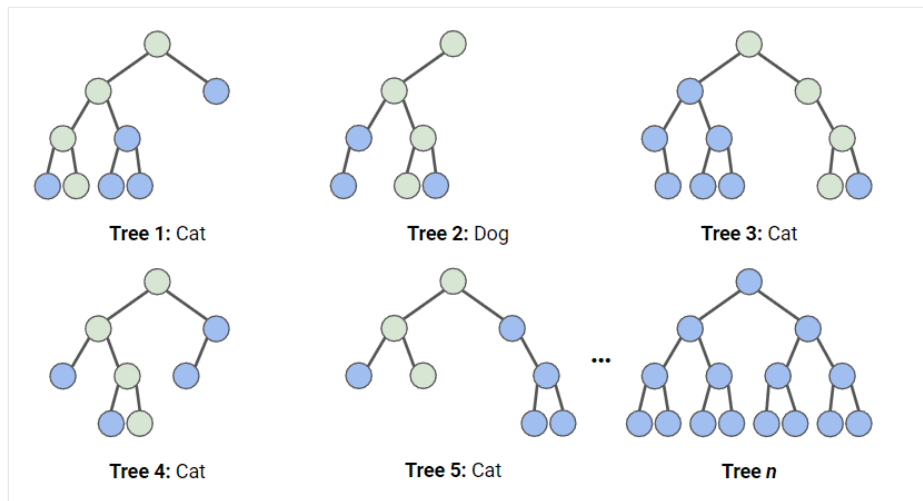
Domine as principais habilidades em R para se tornar um cientista de aprendizado de máquina

Como funciona a classificação do Random Forest

Imagine que você tenha um problema complexo para resolver e reúna um grupo de especialistas de diferentes áreas para dar suas opiniões. Cada especialista fornece sua opinião com base em seu conhecimento e experiência. Em seguida, os especialistas votariam para chegar a uma decisão final.

Em uma classificação de floresta aleatória, várias árvores de decisão são criadas usando diferentes subconjuntos aleatórios de dados e recursos. Cada árvore de decisão é como um especialista, fornecendo sua opinião sobre como classificar os dados. As previsões são feitas calculando a previsão para cada árvore de decisão e, em seguida, pegando o resultado mais popular. (Para regressão, as previsões usam uma técnica de média).

No diagrama abaixo, temos uma floresta aleatória com n árvores de decisão, e mostramos as 5 primeiras, juntamente com suas previsões ("Cachorro" ou "Gato"). Cada árvore é exposta a um número diferente de recursos e a uma amostra diferente do conjunto de dados original e, portanto, cada árvore pode ser diferente. Cada árvore faz uma previsão. Observando as primeiras 5 árvores, podemos ver que 4/5 previram que a amostra era um gato. Os círculos verdes indicam um caminho hipotético que a árvore seguiu para chegar à sua decisão. A floresta aleatória contaria o número de previsões das árvores de decisão para Gato e para Cachorro e escolheria a previsão mais popular.



O conjunto de dados

Esse conjunto de dados consiste em campanhas de marketing direto de uma instituição bancária portuguesa usando chamadas telefônicas. As campanhas tinham como objetivo vender assinaturas de um depósito bancário a prazo. Vamos armazenar esse conjunto de dados em uma variável chamada `bank_data`.

As colunas que usaremos são:

- `age` : A idade da pessoa que recebeu a chamada telefônica
- `default` : Se a pessoa tem crédito inadimplente
- `cons.price.idx` : Pontuação do índice de preços ao consumidor no momento da chamada
- `cons.conf.idx` Índice de confiança do consumidor no momento da chamada
- `y` : Se a pessoa se inscreveu (é isso que estamos tentando prever)

Importação de pacotes

Os seguintes pacotes e funções são usados neste tutorial:

```
# Data Processing
import pandas as pd
import numpy as np

# Modelling
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, ConfusionMatrixDisplay
from sklearn.model_selection import RandomizedSearchCV, train_test_split
from scipy.stats import randint

# Tree Visualisation
from sklearn.tree import export_graphviz
```



```
from IPython.display import Image
import graphviz
```



POWERED BY

Fluxo de trabalho de florestas aleatórias

Para ajustar e treinar esse modelo, seguiremos o infográfico [Fluxo de trabalho de aprendizado de máquina](#); no entanto, como nossos dados são bastante limpos, não executaremos todas as etapas. Faremos o seguinte:

- Engenharia de recursos
- Dividir os dados
- Treinar o modelo
- Ajuste de hiperparâmetros
- Avaliar o desempenho do modelo

Pré-processamento de dados para Random Forests

Os modelos baseados em árvores são muito mais robustos em relação a outliers do que os modelos lineares, e não precisam que as variáveis sejam normalizadas para funcionar. Dessa forma, precisamos fazer muito pouco pré-processamento em nossos dados.

- Mapearemos nossa coluna "default", que contém `no` e `yes`, para `0` e `1`, respectivamente. Neste exemplo, trataremos os valores de `unknown` como `no`.
- Também mapearemos nosso alvo, `y`, para `1` e `0`.

```
bank_data['default'] = bank_data['default'].map({'no':0,'yes':1,'unknown':0})
bank_data['y'] = bank_data['y'].map({'no':0,'yes':1})
```



POWERED BY

Dividindo os dados

Ao treinar qualquer modelo de aprendizado supervisionado, é importante dividir os dados em dados de treinamento e de teste. Os dados de treinamento são usados para ajustar o modelo. O algoritmo usa os dados de treinamento para aprender a relação entre os recursos e o alvo. Os dados de teste são usados para avaliar o desempenho do modelo.

O código abaixo divide os dados em variáveis separadas para os recursos e o destino e, em seguida, divide-os em dados de treinamento e de teste.

```
# Split the data into features (X) and target (y)
X = bank_data.drop('y', axis=1)
y = bank_data['y']

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```



POWERED BY

Ajuste e avaliação do modelo

Primeiro, criamos uma instância do modelo Random Forest, com os parâmetros padrão. Em seguida, ajustamos isso aos nossos dados de treinamento. Passamos os recursos e a variável de destino para que o modelo possa aprender.

```
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```



POWERED BY

Neste ponto, temos um modelo treinado do Random Forest, mas precisamos descobrir se ele está fazendo previsões precisas.

```
y_pred = rf.predict(X_test)
```



POWERED BY

A maneira mais simples de avaliar esse modelo é usando a precisão; verificamos as previsões em relação aos valores reais no conjunto de testes e contamos quantos o modelo acertou.

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```



POWERED BY

Saída:

```
Accuracy: 0.888
```



POWERED BY

Essa é uma pontuação muito boa! No entanto, podemos ter um desempenho melhor ao otimizar nossos hiperparâmetros.

Visualizando os resultados

Podemos usar o código a seguir para visualizar nossas três primeiras árvores.

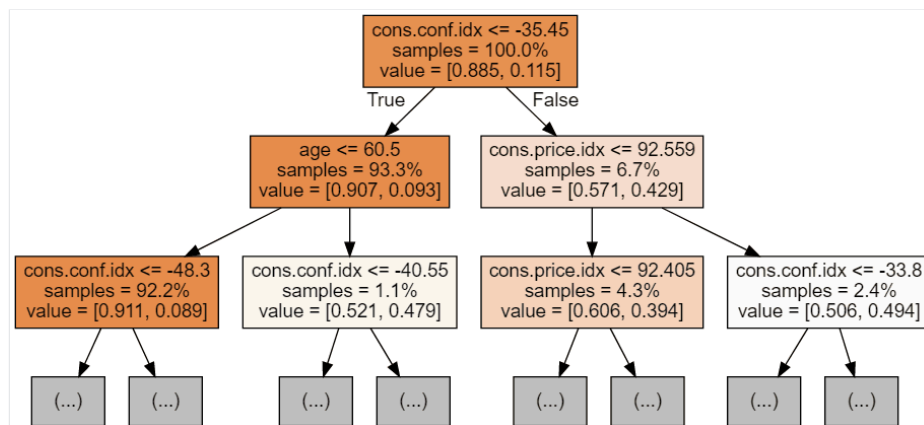
```
# Export the first three decision trees from the forest
```

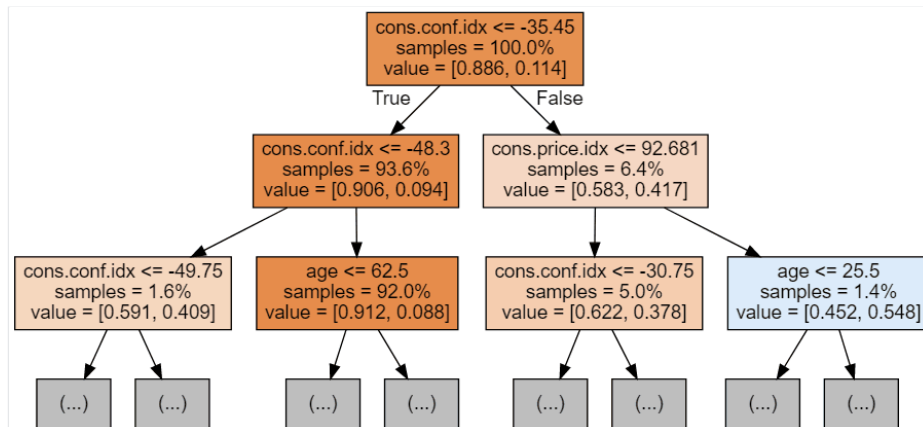
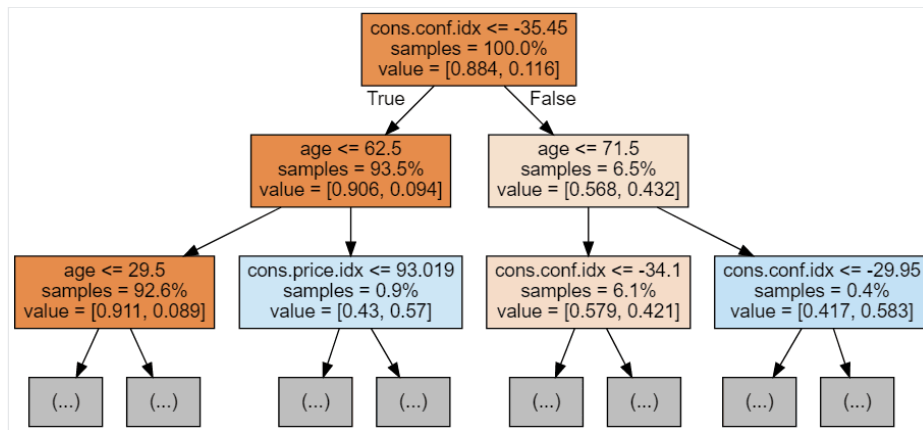


```
for i in range(3):
    tree = rf.estimators_[i]
    dot_data = export_graphviz(tree,
                                feature_names=X_train.columns,
                                filled=True,
                                max_depth=2,
                                impurity=False,
                                proportion=True)
    graph = graphviz.Source(dot_data)
    display(graph)
```



POWERED BY





Cada imagem de árvore é limitada a mostrar apenas os primeiros nós. Essas árvores podem ficar muito grandes e difíceis de visualizar. As cores representam a classe majoritária de cada nó (caixa), com vermelho indicando maioria 0 (sem assinatura) e azul indicando maioria 1 (assinatura). As cores ficam mais escuras quanto mais próximo o nó estiver de ser totalmente 0 ou 1. Cada nó também contém as seguintes informações:

1. O nome e o valor da variável usados para a divisão
2. A porcentagem do total de amostras em cada divisão
3. A % de divisão entre as classes em cada divisão

Ajuste de hiperparâmetros

O código abaixo usa o **RandomizedSearchCV** do Scikit-Learn, que pesquisará aleatoriamente os parâmetros dentro de um intervalo por hiperparâmetro. Definimos os hiperparâmetros a serem usados e seus intervalos no dicionário `param_dist`. No nosso caso, estamos usando:

- **n_estimators**: o número de árvores de decisão na floresta. O aumento desse hiperparâmetro geralmente melhora o desempenho do modelo, mas também aumenta o custo computacional do treinamento e da previsão.
- **max_depth**: a profundidade máxima de cada árvore de decisão na floresta. A definição de um valor mais alto para `max_depth` pode levar a um ajuste excessivo, enquanto a definição de um valor muito baixo pode levar a um ajuste insuficiente.

```

param_dist = {'n_estimators': randint(50,500),
              'max_depth': randint(1,20)}

# Create a random forest classifier
rf = RandomForestClassifier()

# Use random search to find the best hyperparameters
rand_search = RandomizedSearchCV(rf,
                                param_distributions = param_dist,
                                n_iter=5,
                                cv=5)
  
```



```
# Fit the random search object to the data
rand_search.fit(X_train, y_train)
```



POWERED BY

RandomizedSearchCV treinará muitos modelos (definidos por `n_iter_` e salvará cada um deles como variáveis, o código abaixo cria uma variável para o melhor modelo e imprime os hiperparâmetros. Nesse caso, não passamos um sistema de pontuação para a função, portanto, o padrão é a precisão. Essa função também usa validação cruzada, o que significa que ela divide os dados em cinco grupos de tamanho igual e usa 4 para treinar e 1 para testar o resultado. Ele percorrerá cada grupo e fornecerá uma pontuação de precisão, cuja média será calculada para encontrar o melhor modelo.

```
# Create a variable for the best model
best_rf = rand_search.best_estimator_

# Print the best hyperparameters
print('Best hyperparameters:', rand_search.best_params_)
```



POWERED BY

Saída:

```
Best hyperparameters: {'max_depth': 5, 'n_estimators': 260}
```



POWERED BY

Mais métricas de avaliação

Vamos dar uma olhada na matriz de confusão. Isso representa o que o modelo previu em relação ao que foi a previsão correta. Você pode usar isso para entender a troca entre falsos positivos (canto superior direito) e falsos negativos (canto inferior esquerdo). Podemos traçar a matriz de confusão usando esse código:

```
# Generate predictions with the best model
y_pred = best_rf.predict(X_test)

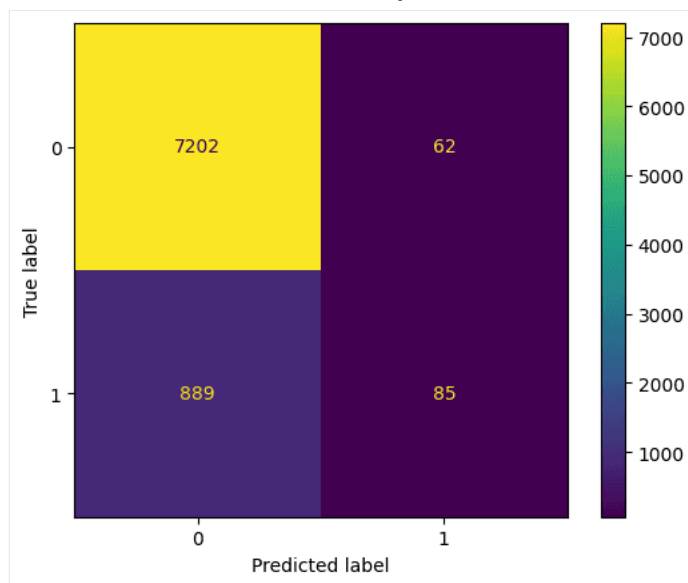
# Create the confusion matrix
cm = confusion_matrix(y_test, y_pred)

ConfusionMatrixDisplay(confusion_matrix=cm).plot();
```



POWERED BY

Saída:



Também devemos avaliar o melhor modelo com exatidão, precisão e recuperação (observe que seus resultados podem ser diferentes devido à randomização)

```
y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
```



POWERED BY

Saída:

Accuracy: 0.885

Precision: 0.578

Recall: 0.0873



POWERED BY

O código abaixo representa a importância de cada recurso, usando a pontuação interna do modelo para encontrar a melhor maneira de dividir os dados em cada árvore de decisão.

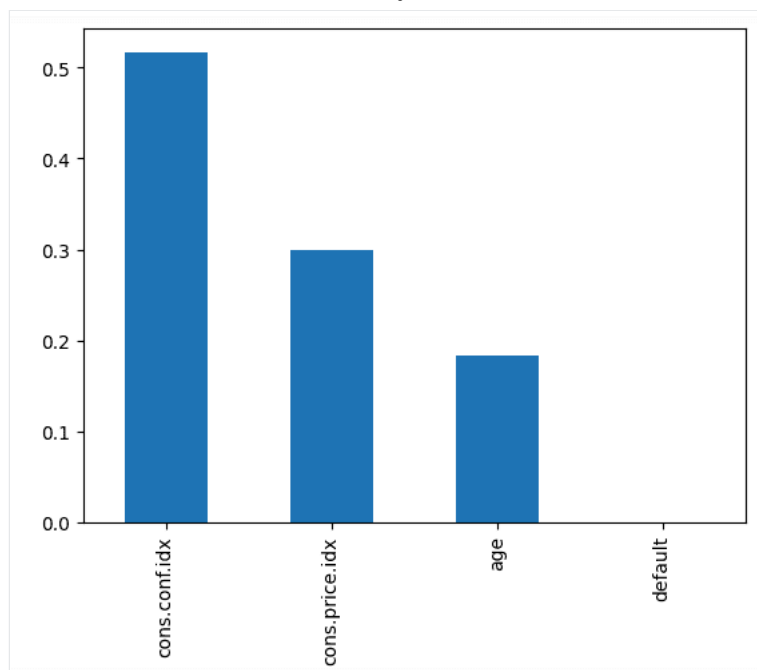
```
# Create a series containing feature importances from the model and feature names from the training data
feature_importances = pd.Series(best_rf.feature_importances_, index=X_train.columns).sort_values(ascending=False)

# Plot a simple bar chart
feature_importances.plot.bar();
```



POWERED BY

Isso nos diz que o índice de confiança do consumidor, no momento da chamada, foi o maior indicador da assinatura da pessoa.



Leve-o para o próximo nível

- Para começar a usar o aprendizado de máquina supervisionado em Python, faça o curso [Aprendizado supervisionado com o scikit-learn](#).
- Para saber mais, o uso de florestas aleatórias (e outros modelos de aprendizado de máquina baseados em [árvores](#)) é abordado com mais profundidade em [Aprendizado de máquina com modelos baseados em árvores em Python](#) e [Métodos de conjunto em Python](#).
- Faça o download da [folha de dicas do scikit-learn](#) para que você tenha uma referência útil do código abordado neste tutorial.

Treinar mais pessoas?

Dê à sua equipe acesso à plataforma completa do DataCamp para empresas.

Para uma solução sob medida, [agende uma demonstração](#).